

Demystifying Large Language Models

How Transformers Actually Work

Presented by: Eman Elrefai

Agenda Overview

1

Recap & Context Bridge

Connecting to the previous session — simple models vs. today's powerful LLMs.

2

The Bigram Model

The simplest language model in history: what “predicting the next word” really means.

3

Word Embeddings

How computers see words as points in a vast geometric space.

4

Self-Attention

Inside the Transformer — the Query, Key, Value equation.

5

Full Transformer Architecture

Putting it together: Encoder and Decoder.

6

Live Coding

Google Colab: implementing everything in interactive Python.

Recap & Context Bridge

Next-Token Prediction

Karthik's joke generator was built on one idea: guessing “what is the next word?” — the foundation behind every LLM in the world.

“Goldfish Memory”

A simple model only looks at the last word or two. Give it a long sentence, and by the end it has forgotten how it started — losing the context completely.

Today: how the Transformer gave models an “iron memory,” connecting the first word of a book to the last.

The Bigram Model

Pure Probability: The model asks: based on training data, what word is most likely to come next?

WhatsApp Keyboard: Type “Happy” and your keyboard suggests “Birthday,” “New,” or “Anniversary” — a statistical table, not real intelligence.

Blind to Context: For “He is not Artificial _____,” it still predicts “Intelligence” — it cannot see the word “not.”

$$P(\text{Word}_i \mid \text{Word}_{i-1})$$

Probability of the next word,
given only the one before it.

Tokenization & Vocabulary

Computers Are Numeric: Text must become numbers before any math can happen.

Tokens: A sentence is chopped into pieces — full words, sub-words, or single characters.

Vocabulary IDs: Each token gets a unique ID from a huge dictionary of every token in the world.

“Transformer”

```
[ 'Transform', 'er' ]
```

ID 4521

ID 89

The sentence becomes an array of numbers, ready for math.

Word Embeddings

Words to Math: Every token is projected into a vector — a massive geometric space (768, even 4096+ dimensions).

Semantic Proximity: Words with similar meaning, like “Dog” and “Cat,” land close together in that space.

Math on Meaning: $\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$. The computer isn't reading numbers anymore — it's computing meaning.

King – Man + Woman

$\approx$ Queen

The most famous example of semantic vector arithmetic.

Positional Encoding

Parallel, Not Sequential: The Transformer reads an entire sentence at once — a huge speed advantage.

Lost Order: “The dog ate the fish” and “The fish ate the dog” use identical words — the model can't tell them apart.

The Fix: Add a Positional Encoding vector to each Word Embedding, so every word carries “what I mean” and “where I am.”

Embedding + Position

$$\begin{array}{c} \text{Word Embedding} \\ + \\ \text{Positional Encoding} \\ = \\ \text{Full Input Vector} \end{array}$$

Limits of RNNs & LSTMs

The Bottleneck

An RNN reads word by word, compressing everything into one tiny Hidden State — like summarizing an entire book on a single page. Early details vanish (the Vanishing Gradient problem).

Sequential Processing

Each word must wait for the last one to be read. GPUs are built for massive parallel work — but RNNs force them to work step-by-step, making training painfully slow.

Self-Attention Concept

In 2017, Google researchers published “Attention Is All You Need,” introducing Self-Attention.

The Cocktail Party Effect

In a loud, crowded room, you can still focus entirely on your friend's voice. Self-Attention lets every word “look” at every other word and decide what deserves focus.

“It” — Who?

“The animal didn't cross the street because it was too tired.”
Self-Attention gives “it” a strong score with “animal,” not “street” — the word carries the whole sentence's meaning.

Query, Key, Value Intuition

Every word becomes three vectors — an idea borrowed from search engines.

Query

What you type in the YouTube search bar — your question or request.

Key

Every video title in the database — what's on offer to match against.

Value

The actual video you get to watch — the content itself.

When a Query matches a Key, the word pulls in that Value to build its own meaning.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V$$

1

Multiply

Q by K^T (dot product) to get raw attention Scores.

2

Scale

Divide by $\sqrt{d_k}$ so numbers stay stable and Softmax doesn't break.

3

Softmax

Convert messy scores into percentages that sum to 100%.

4

Weight

Multiply those percentages by V to extract the important data.

Multi-Head Attention

Parallel Perspectives: Why calculate Attention only once? The model runs multiple Heads at the same time.

Different Focus: Head 1 might track grammar, Head 2 sentiment, Head 3 pronoun references — each views the sentence from a different angle.

Combining Insights: All Heads are concatenated and passed through a linear layer for a deep, multi-dimensional understanding.

3 Heads, 3 Views

Head 1: Grammar

Head 2: Sentiment

Head 3: Pronouns



Concatenate + Linear

Feed-Forward Networks & Residual Connections

Feed-Forward Network

A standard neural net applied to each word individually. Attention finds relationships between words — the FFN adds non-linearity and digs into deeper, more complex patterns.

Residual Connections

Like an overpass bridge over traffic: $x + \text{SubLayer}(x)$. The original signal skips ahead, so information never gets lost deep in the layers — training becomes faster and more stable.

The Full Transformer Architecture

The Encoder

Takes the input text, analyzes it, and understands the context perfectly — converting it into a rich mathematical representation. It only understands; it doesn't speak.

The Decoder

Takes the Encoder's rich understanding and generates new output text word by word — like translating into another language.

Encoder-only vs. Decoder-only

Encoder-only (BERT)

Reads bidirectionally — the whole text at once, like a reading-comprehension exam. Great for classification & sentiment, poor at generating text.

Decoder-only (GPT, Llama, Gemma)

Generates left-to-right, one token at a time. The kings of conversation, text generation, and code — today's superstar architecture.

Encoder-Decoder (T5)

Uses both together. Exceptionally good for translation and summarizing large articles.

Autoregressive Generation & Masking

Autoregressive Loop: Generate a token, feed it back in, generate the next — until an <eos> token says “I’m done.”

The Cheating Problem: During training, the model can already see the whole sentence — so it needs to be stopped from peeking ahead.

Causal Masking: Future words get $-\infty$ in the Attention matrix. After Softmax, their probability becomes zero — only past and present are visible.

token → model → next token
↑ _____ ↓

Loops until <eos>

Sampling Strategy: Temperature & Top-k/p

Temperature

Low (0.2): precise & rational — great for code or facts. High (0.8–1.0): creative and a bit unpredictable — great for stories.

Top-K

Sort words by probability, keep only the top K (e.g. 50), and sample randomly from that shortlist.

Top-P (Nucleus)

Add up probabilities from the top until they reach a threshold (e.g. 90%), and sample only from that group.

Interactive Coding: What We'll Build

 [Open the Colab Notebook](#)

1

The Tokenizer & Embeddings

Convert our text into sequences of numbers and initialize the mathematical embedding tables.

2

The Attention Block

Implement the Query, Key, and Value matrices from scratch in PyTorch to see the math in action.

3

The Feed-Forward Layer

Add the deep neural network layer that processes the output of the attention block.

4

Putting It Together

Combine these blocks into a true miniature Transformer and train it on a small dataset!

Inside the Code (Interactive Coding)

Inside SelfAttentionHead — the Q, K, V equation, four lines of PyTorch:

```
wei = q @ k.transpose(-2, -1) * C**-.5  
wei = wei.masked_fill(tril == 0, float('-inf'))  
wei = F.softmax(wei, dim=-1)  
out = wei @ v
```

Scaling by $\sqrt{d_k}$ → Causal Masking → Softmax → weighted sum with V — the brain of an LLM, right here.

Google Cloud Skills Boost

Gen AI Path

Take the free “Introduction to Large Language Models” course to continue your journey.

Hands-On Labs

Practice fine-tuning and deploying models safely with real Google Cloud tools in a sandboxed environment.

Earn Badges

Complete learning paths to earn official Google Cloud skill badges for your LinkedIn profile.

Questions?

Thank you for joining the workshop!

`eman.lotfy.elrefai@gmail.com`

[Medium](#) · [LinkedIn](#)



Feedback